



Bikkad IT Institute

NumPy Fundamentals Notes

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

NumPy Fundamentals Notes

Arrays, Operations, Indexing and Reshaping (Compact Edition)

Prepared for Bikkad IT Institute Students | Level: Beginner to Intermediate | Environment:
Jupyter Notebook or Google Colab

Topic Index

- 1. What is NumPy?
- 2. NumPy Arrays vs Python Sequences
- 3. Creating NumPy Arrays
- 4. Array Attributes
- 5. Changing Datatype
- 6. Array Operations
- 7. Array Functions
- 8. Indexing and Slicing
- 9. Iterating
- 10. Reshaping
- 11. Stacking
- 12. Splitting

1. What is NumPy?

NumPy is the fundamental package for scientific and numerical computing in Python. It provides a fast, memory-efficient multidimensional array object called ndarray, along with a large collection of routines for mathematical, logical, and statistical operations, all without writing explicit Python loops.

Syntax

```
import numpy as np
np.array(object)
```

OUR LOCATIONS

Example 1: Creating 1D, 2D and 3D ndarrays

```
import numpy as np
a = np.array([1, 2, 3])
grid = np.array([[1, 2, 3], [4, 5, 6]])
print(a, '-> ndim', a.ndim)
print(grid, '-> ndim', grid.ndim)
```

Output / Result

```
[1 2 3] -> ndim 1
[[1 2 3]
 [4 5 6]] -> ndim 2
```

Explanation

np is the conventional alias for numpy. The same np.array() constructor builds 1D, 2D, or higher-dimensional arrays depending on how the input list is nested.

Example 2: Built-in mathematical routines

```
values = np.array([1, 4, 9, 16, 25])
print(np.sqrt(values))
print(np.sum(values))
```

Output / Result

```
[1.  2.  3.  4.  5.]
55
```

Explanation

NumPy ships with a large library of ready-made mathematical functions (ufuncs) that operate on an entire array at once - the 'assortment of routines' the library is known for.

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 3: Why loops are avoided - vectorized speed

```
import time
size = 2_000_000
py_list = list(range(size))
np_arr = np.arange(size)
start = time.time()
py_result = [x * 2 for x in py_list]
py_time = time.time() - start
start = time.time()
np_result = np_arr * 2
np_time = time.time() - start
print('Python loop time :', round(py_time, 4))
print('NumPy vector time :', round(np_time, 4))
```

Output / Result

```
Python loop time : 0.0621
NumPy vector time : 0.0052
```

Explanation

NumPy performs the multiplication inside optimized C code (vectorization), which is dramatically faster than looping over a Python list element by element.

2. NumPy Arrays vs Python Sequences

A NumPy ndarray looks similar to a Python list but behaves very differently underneath - in fixed size, in how it stores data types, and in memory usage.

Syntax

```
list.append(value) # Python list - grows dynamically
np.append(array, value) # NumPy array - returns a NEW array
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 1: Fixed size vs dynamic size

```
py_list = [1, 2, 3]
py_list.append(4)
print('List after append:', py_list)
arr = np.array([1, 2, 3])
new_arr = np.append(arr, 4)
print('Original array :', arr)
print('New array :', new_arr)
```

Output / Result

```
List after append: [1, 2, 3, 4]
Original array : [1 2 3]
New array : [1 2 3 4]
```

Explanation

A Python list grows in place. An ndarray has a fixed size at creation, so `np.append()` does not modify `arr`; it builds and returns a brand-new array.

Example 2: Homogeneous data type vs mixed types

```
mixed_list = [1, 'two', 3.0, True]
print([type(x).__name__ for x in mixed_list])
mixed_arr = np.array([1, 'two', 3.0, True])
print(mixed_arr)
print(mixed_arr.dtype)
```

Output / Result

```
['int', 'str', 'float', 'bool']
['1' 'two' '3.0' 'True']
<U32
```

Explanation

A Python list freely mixes data types. NumPy requires one type per array, so it upcasts every element (here to a common string type) instead of keeping them separate.

OUR LOCATIONS

Example 3: Memory footprint comparison

```
import sys
py_list = list(range(1000))
np_arr = np.arange(1000)
list_bytes = sys.getsizeof(py_list) + sum(
    sys.getsizeof(x) for x in py_list)
print('Python list (approx bytes):', list_bytes)
print('NumPy array (bytes) :', np_arr.nbytes)
```

Output / Result

```
Python list (approx bytes): 36056
NumPy array (bytes) : 8000
```

Explanation

Because every element in an ndarray is the same fixed-width type stored contiguously in memory, NumPy arrays occupy far less memory than an equivalent Python list of boxed objects.

3. Creating NumPy Arrays

NumPy offers several constructors for building arrays: converting existing Python sequences, generating regular numeric ranges, and creating arrays pre-filled with a constant or random values.

Syntax

```
np.array(data, dtype=None)
np.arange(start, stop, step)
np.ones(shape); np.zeros(shape); np.identity(n)
np.random.random(shape); np.linspace(start, stop, num)
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Example 1: np.array with dtype control

```
a = np.array([1, 2, 3])
b = np.array([[1, 2], [3, 4]])
float_arr = np.array([1, 2, 3], dtype=float)
print(a)
print(b)
print(float_arr, float_arr.dtype)
```

Output / Result

```
[1 2 3]
[[1 2]
 [3 4]]
[1. 2. 3.] float64
```

Explanation

The nesting of the input list decides the number of dimensions, and the dtype parameter overrides NumPy's automatic type inference for every element.

Example 2: Ranges with arange and reshape

```
seq = np.arange(1, 11, 2)
print(seq)
grid = np.arange(16).reshape(2, 2, 2, 2)
print(grid.shape)
```

Output / Result

```
[1 3 5 7 9]
(2, 2, 2, 2)
```

Explanation

np.arange(start, stop, step) generates values like Python's range(), and .reshape() rearranges that flat sequence into any shape with a matching total element count.

OUR LOCATIONS

Example 3: Pre-filled and random arrays

```
print(np.ones((2, 3)))
print(np.identity(3))
np.random.seed(42)
print(np.round(np.random.random((2, 3)), 2))
print(np.linspace(-10, 10, 5, dtype=int))
```

Output / Result

```
[[1.  1.  1.]
 [1.  1.  1.]]
[[1.  0.  0.]
 [0.  1.  0.]
 [0.  0.  1.]]
[[0.37 0.95 0.73]
 [0.6  0.16 0.16]]
[-10 -5  0  5 10]
```

Explanation

`np.ones()` and `np.identity()` build constant or identity matrices; `np.random.random()` generates uniform values for simulations; `np.linspace(start, stop, num)` generates an exact count of evenly spaced points.

4. Array Attributes

Every `ndarray` carries metadata describing its shape and storage: `ndim`, `shape`, `size`, `itemsize`, and `dtype`. These are essential for debugging and for writing code that adapts to the data it receives.

Syntax

```
a.ndim; a.shape; a.size; a.itemsize; a.dtype
```

Example 1: ndim and shape

```
a1 = np.arange(10, dtype=np.int32)
a2 = np.arange(12, dtype=float).reshape(3, 4)
a3 = np.arange(8).reshape(2, 2, 2)
print('a1:', a1.ndim, a1.shape)
print('a2:', a2.ndim, a2.shape)
print('a3:', a3.ndim, a3.shape)
```

Output / Result

```
a1: 1 (10,)
a2: 2 (3, 4)
a3: 3 (2, 2, 2)
```

Explanation

`ndim` reports how many axes an array has, and `shape` returns a tuple with the length along each of those axes in order

OUR LOCATIONS

Example 2: size and itemsize

```
print('a1:', a1.size, 'elems,', a1.itemsize, 'bytes each')
print('a2:', a2.size, 'elems,', a2.itemsize, 'bytes each')
print('a3:', a3.size, 'elems,', a3.itemsize, 'bytes each')
```

Output / Result

```
a1: 10 elems, 4 bytes each
a2: 12 elems, 8 bytes each
a3: 8 elems, 8 bytes each
```

Explanation

size is the product of every dimension in shape (total element count); itemsize reports how many bytes each individual element occupies, based on its dtype.

Example 3: dtype - the element data type

```
print('a1 dtype:', a1.dtype)
print('a2 dtype:', a2.dtype)
print('a3 dtype:', a3.dtype)
```

Output / Result

```
a1 dtype: int32
a2 dtype: float64
a3 dtype: int64
```

Explanation

dtype identifies exactly how each value is stored. a1 was created as int32, a2 as float, and a3 falls back to NumPy's default integer type.

5. Changing Datatype

Once an array exists, its data type can be converted with `.astype()`, useful for controlling memory usage or preparing data for a function that expects a specific type.

Syntax

```
a.astype(new_dtype)
```

Example 1: Float <-> integer conversion

```
prices = np.array([10.9, 20.2, 30.8])
whole = prices.astype(int)
back_to_float = whole.astype(np.float64)
print(prices)
print(whole)
print(back_to_float)
```

Output / Result

```
[10.9 20.2 30.8]
[10 20 30]
[10. 20. 30.]
```

Explanation

Converting float to int truncates the decimal part (it does not round); converting back to float simply adds the decimal point again.

Example 2: Numbers to strings

```
ids = np.array([101, 102, 103])
as_text = ids.astype(str)
print(as_text, as_text.dtype)
```

Output / Result

```
['101' '102' '103'] <U21
```

Explanation

`astype(str)` converts each numeric value into its text representation, handy before joining numbers into labels or file names.

Example 3: Numbers to boolean

```
flags = np.array([0, 5, -3, 0, 1])
as_bool = flags.astype(bool)
print(as_bool)
```

Output / Result

```
[False True True False True]
```

Explanation

When converting to bool, zero becomes False and any non-zero number, positive or negative, becomes True.

OUR LOCATIONS

6. Array Operations

NumPy overloads Python's arithmetic and comparison operators so they work element-wise. A scalar operation applies the same value to every element, while a vector operation combines two arrays position by position.

Syntax

```
a + scalar; a ** scalar; a == value; a > value  
a1 + a2; a1 * a2; a1 ** a2
```

Example 1: Scalar arithmetic operations

```
a1 = np.arange(12).reshape(3, 4)  
print(a1)  
print(a1 ** 2)
```

Output / Result

```
[[ 0  1  2  3]  
 [ 4  5  6  7]  
 [ 8  9 10 11]]  
[[ 0  1  4  9]  
 [16 25 36 49]  
 [64 81 100 121]]
```

Explanation

Every element of a1 is independently raised to the power of 2; the operator is broadcast across the whole array with no explicit loop.

Example 2: Scalar and vector relational operations

```
a2 = np.arange(12, 24).reshape(3, 4)  
print(a2 == 15)  
print(a1 > 5)  
print(a1 < a2)
```

Output / Result

```
[[False False False True]  
 [False False False False]  
 [False False False False]]  
[[False False False False]  
 [False False True True]  
 [ True True True True]]  
[[ True True True True]  
 [ True True True True]  
 [ True True True True]]
```

Explanation

Comparison operators return a Boolean array of the same shape - against a scalar, or against a second array of matching shape, marking True wherever the condition holds.

OUR LOCATIONS

Example 3: Vector arithmetic between two arrays

```
print(a1 + a2)
print(a1 * a2)
small_a = np.array([[1, 2], [3, 4]])
small_b = np.array([[3, 1], [2, 2]])
print(small_a ** small_b)
```

Output / Result

```
[[12 14 16 18]
 [20 22 24 26]
 [28 30 32 34]]
[[ 0 13 28 45]
 [ 64 85 108 133]
 [160 189 220 253]]
[[ 1 2]
 [ 9 16]]
```

Explanation

When two arrays share the same shape, arithmetic operators combine them element by element. Keeping the numbers small for the power example also avoids silent integer overflow from large exponents.

7. Array Functions

Beyond basic operators, NumPy ships with aggregation functions (sum, mean, max...), mathematical ufuncs (sin, log, exp...), and linear-algebra helpers such as the dot product, all built to run efficiently across whole arrays.

Syntax

```
np.sum(a, axis=None); np.mean(a); np.std(a)
np.sin(a); np.log(a); np.exp(a)
np.dot(a, b)
```

OUR LOCATIONS

Example 1: Aggregations and statistical summaries

```
np.random.seed(1)
a1 = np.round(np.random.random((3, 3)) * 100)
print(a1)
print('Column sums:', np.sum(a1, axis=0))
print('Row means :', np.round(np.mean(a1, axis=1), 2))
print('Std :', round(np.std(a1), 2))
```

Output / Result

```
[[42. 72. 0.]
 [30. 15. 9.]
 [19. 35. 40.]]
Column sums: [ 91. 122. 49.]
Row means : [38. 18. 31.33]
Std : 20.31
```

Explanation

axis=0 collapses down the rows (one result per column) while axis=1 collapses across the columns (one result per row); omitting axis reduces the whole array to a single value.

Example 2: Trigonometric, log, exp and rounding

```
vals = np.array([1, 2.3, 4.7, 8])
print(np.round(np.sin(vals), 2))
print(np.round(np.log(vals), 2))
print(np.floor(vals), np.ceil(vals))
```

Output / Result

```
[ 0.84 0.75 -1.  0.99]
[0.  0.83 1.55 2.08]
[1.  2.  4.  8.] [1.  3.  5.  8.]
```

Explanation

Trigonometric and exponential ufuncs apply independently to every element; floor always rounds down and ceil always rounds up to the nearest whole number.

OUR LOCATIONS

Example 3: Dot product and matrix multiplication

```
a2 = np.arange(12).reshape(3, 4)
a3 = np.arange(12, 24).reshape(4, 3)
print(np.dot(a2, a3))
```

Output / Result

```
[[114 120 126]
 [378 400 422]
 [642 680 718]]
```

Explanation

np.dot() performs true matrix multiplication for 2D arrays: the inner dimensions (4 and 4 here) must match, and the result takes the outer shape (3 x 3).

8. Indexing and Slicing

NumPy uses zero-based indexing, exactly like Python lists. For multidimensional arrays, a single index expression can select an axis-by-axis position, and slices follow the familiar start:stop:step pattern with stop excluded.

Syntax

```
a[row, column] or a[axis0, axis1, axis2]
a[start:stop:step]
```

Example 1: 1D and 2D indexing

```
a1 = np.arange(10)
a2 = np.arange(12).reshape(3, 4)
print(a1[0], a1[-1], a1[5])
print(a2[1, 0], a2[2, 3])
```

Output / Result

```
0 9 5
4 11
```

Explanation

A single index picks one element; a2[row, column] selects one element of a 2D array directly, avoiding the nested-bracket syntax plain Python lists need.

OUR LOCATIONS

Example 2: 3D indexing

```
a3 = np.arange(8).reshape(2, 2, 2)
print(a3)
print(a3[1, 0, 1])
print(a3[1, 1, 0])
```

Output / Result

```
[[[0 1]
   [2 3]]

  [[4 5]
   [6 7]]]
5
6
```

Explanation

A 3D array needs three indices: the first selects the outer block, the second the row within it, and the third the column within that row.

Example 3: 1D and 2D slicing patterns

```
print(a1[2:8:2])
print(a1[::-1])
print(a2[0:2, 1::2])
print(a2[:, 1::2])
```

Output / Result

```
[2 4 6]
[9 8 7 6 5 4 3 2 1 0]
[[1 3]
 [5 7]]
[[ 1 3]
 [ 9 11]]
```

Explanation

start:stop:step works as with Python lists, with stop never included and a step of -1 reversing the array. Each axis of a 2D array can be sliced independently in one bracket, separated by a comma.

9. Iterating

NumPy arrays can be looped over directly, but what each iteration returns depends on the number of dimensions. For a fully flat, element-by-element walk regardless of shape, `np.nditer` is the dedicated tool.

Syntax

```
for x in a: ...
for x in np.nditer(a): ...
```

OUR LOCATIONS

Example 1: Iterating 1D and 2D arrays

```
a1 = np.arange(5)
for i in a1:
    print(i, end=' ')
print()
a2 = np.arange(12).reshape(3, 4)
for row in a2:
    print(row)
```

Output / Result

```
0 1 2 3 4
[0 1 2 3]
[4 5 6 7]
[ 8 9 10 11]
```

Explanation

Looping over a 1D array yields one scalar at a time; looping over a 2D array yields one full row (a 1D sub-array) per iteration, not one scalar element.

Example 2: Iterating a 3D array

```
a3 = np.arange(8).reshape(2, 2, 2)
for block in a3:
    print(block)
```

Output / Result

```
[[0 1]
 [2 3]]
[[4 5]
 [6 7]]
```

Explanation

Looping over a 3D array yields one entire 2D block per iteration; the pattern generalizes so each loop peels off the outermost axis.

Example 3: Flattened iteration with np.nditer

```
for value in np.nditer(a3):
    print(value, end=' ')
```

Output / Result

```
0 1 2 3 4 5 6 7
```

Explanation

np.nditer visits every individual scalar value inside the array in order, regardless of how many dimensions it has - useful when the shape doesn't matter.

OUR LOCATIONS

10. Reshaping

Reshaping changes how an array's elements are arranged into axes without changing the underlying data or the total number of elements. Transpose and ravel are two of the most common reshaping tools.

Syntax

```
a.reshape(new_shape); a.reshape(3, -1)
a.T or np.transpose(a)
a.ravel(); a.flatten()
```

Example 1: Reshape and transpose

```
a = np.arange(1, 7)
m = a.reshape(2, 3)
print(m)
print(m.T)
```

Output / Result

```
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

Explanation

reshape() rearranges the same six values into the requested 2 x 3 grid, and transposing that grid flips rows and columns to produce a 3 x 2 array.

Example 2: Ravel and auto-sized reshape with -1

```
a3 = np.arange(8).reshape(2, 2, 2)
print(a3.ravel())
b = np.arange(12)
print(b.reshape(3, -1))
```

Output / Result

```
[0 1 2 3 4 5 6 7]
[[ 0 1 2 3]
 [ 4 5 6 7]
 [ 8 9 10 11]]
```

Explanation

ravel() collapses an array of any dimension into a single flat 1D sequence. Passing -1 for one dimension in reshape tells NumPy to compute that size automatically.

Example 3: ravel (view) vs flatten (copy)

```
m = np.array([[1, 2], [3, 4]])
r = m.ravel()
r[0] = 99
print('Ravel modified original m:')
print(m)
```

Output / Result

```
Ravel modified original m:
[[99 2]
 [ 3 4]]
```

Explanation

ravel() returns a view sharing memory with the original array whenever possible, so editing it can change the source; flatten() always returns an independent copy.

11. Stacking

Stacking joins two or more arrays together into a single larger array. NumPy provides direction-specific helpers for the common cases, plus a general-purpose function for stacking along a brand-new axis.

Syntax

```
np.hstack((a, b)); np.vstack((a, b))
np.stack((a, b), axis=0); np.concatenate((a, b), axis=0)
```

Example 1: Horizontal and vertical stacking

```
a4 = np.arange(12).reshape(3, 4)
a5 = np.arange(12, 24).reshape(3, 4)
print(np.hstack((a4, a5)))
print(np.vstack((a4, a5)))
```

Output / Result

```
[[ 0  1  2  3 12 13 14 15]
 [ 4  5  6  7 16 17 18 19]
 [ 8  9 10 11 20 21 22 23]]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
```

Explanation

hstack joins arrays side by side along the columns (same row count required); vstack joins them top to bottom along the rows (same column count required).

OUR LOCATIONS

Example 2: Stacking along a brand-new axis

```
b1 = np.array([1, 2, 3])
b2 = np.array([4, 5, 6])
print(np.stack((b1, b2), axis=0))
print(np.stack((b1, b2), axis=1))
```

Output / Result

```
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

Explanation

`np.stack()` introduces a completely new axis to hold the joined arrays, unlike `hstack/vstack` which join along an existing axis.

Example 3: General-purpose concatenate

```
print(np.concatenate((a4, a5), axis=0))
print(np.concatenate((a4, a5), axis=1).shape)
```

Output / Result

```
[[ 0 1 2 3]
 [ 4 5 6 7]
 [ 8 9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
(3, 8)
```

Explanation

`np.concatenate()` is the general form behind `hstack` and `vstack`: `axis=0` behaves like `vstack` and `axis=1` behaves like `hstack` for 2D arrays.

12. Splitting

Splitting is the reverse of stacking: it breaks one array into several smaller arrays, either into equal sections or at custom boundaries that you specify.

Syntax

```
np.hsplit(a, sections); np.vsplit(a, sections)
np.array_split(a, sections); np.split(a, n, axis=k)
```

OUR LOCATIONS

Example 1: Horizontal and vertical split

```
a4 = np.arange(20).reshape(4, 5)
parts = np.hsplit(a4, 5)
print(parts[0].ravel(), parts[1].ravel())
a5 = np.arange(12, 24).reshape(3, 4)
top, bottom = np.vsplit(a5, [1])
print(top)
print(bottom)
```

Output / Result

```
[ 0  5 10 15] [ 1  6 11 16]
[[12 13 14 15]]
[[16 17 18 19]
 [20 21 22 23]]
```

Explanation

hsplit divides an array's columns and vsplit divides its rows; passing a list of index positions (like [1]) splits at that boundary instead of requiring equal-sized sections.

Example 2: Uneven splits with array_split

```
a1 = np.arange(10)
for part in np.array_split(a1, 3):
    print(part)
```

Output / Result

```
[0 1 2 3]
[4 5 6]
[7 8 9]
```

Explanation

array_split() tolerates a section count that does not divide the array evenly, distributing the remaining elements across the earliest sections.

Example 3: Splitting a 3D array along a chosen axis

```
a3 = np.arange(24).reshape(2, 3, 4)
left, right = np.split(a3, 2, axis=2)
print('Left shape:', left.shape)
print('Right shape:', right.shape)
```

Output / Result

```
Left shape : (2, 3, 2)
Right shape: (2, 3, 2)
```

Explanation

np.split() works on any axis of an n-dimensional array; here axis=2 splits the last (depth) axis in half instead of the default first axis.

OUR LOCATIONS